

Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

$\mathbf{X}_1$

$\mathbf{X}_2$

$\mathbf{X}_3$

$\mathbf{Q}_1$

$\mathbf{Q}_2$

$\mathbf{Q}_3$

$\mathbf{Q}_4$

Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

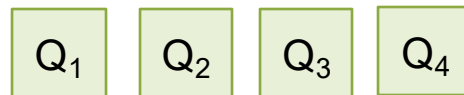
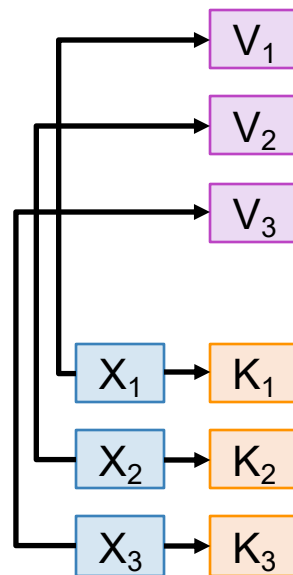
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

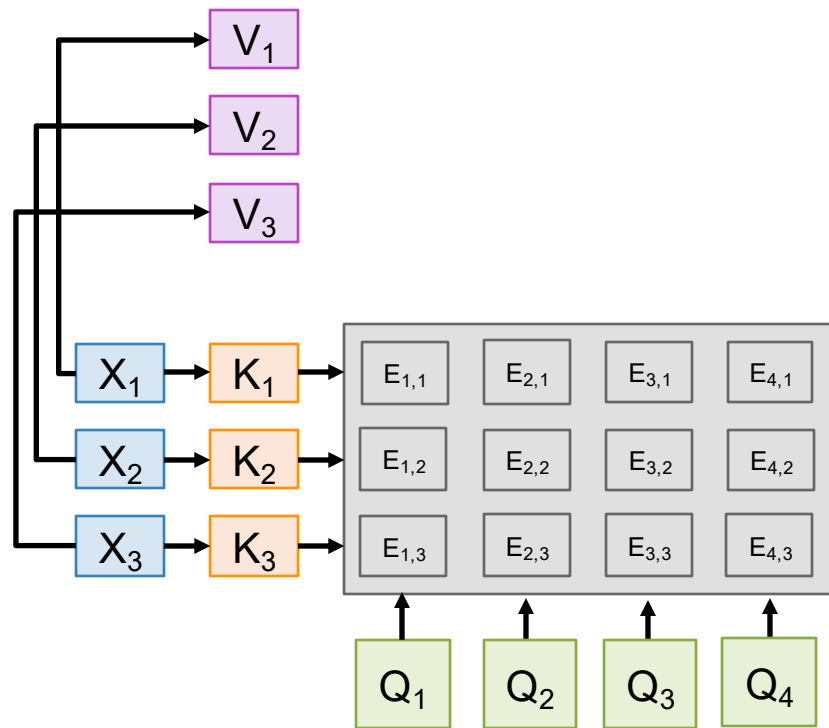
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

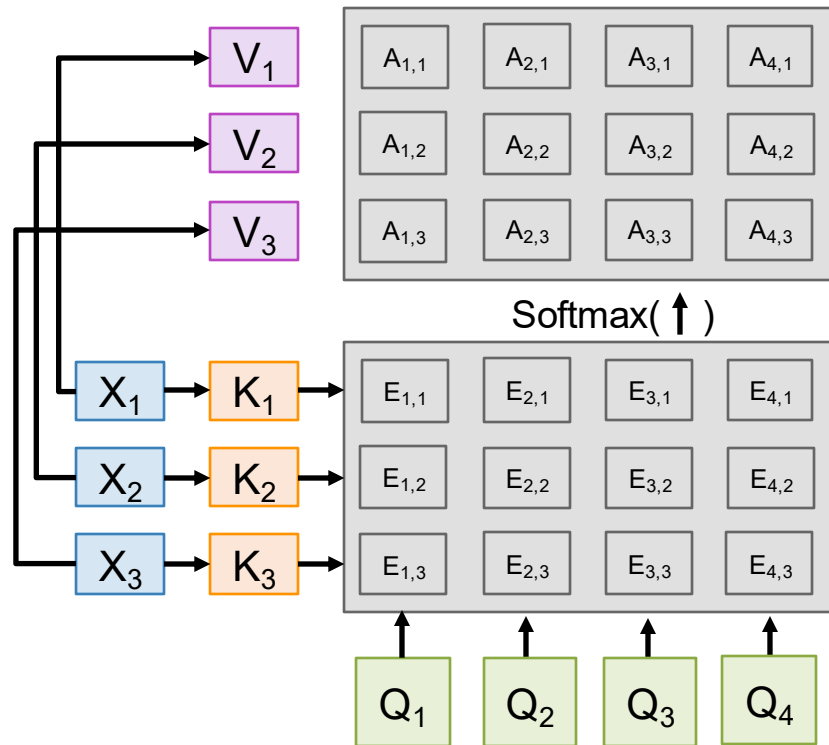
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$

Softmax normalizes each column: each **query** predicts a distribution over the **keys**



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

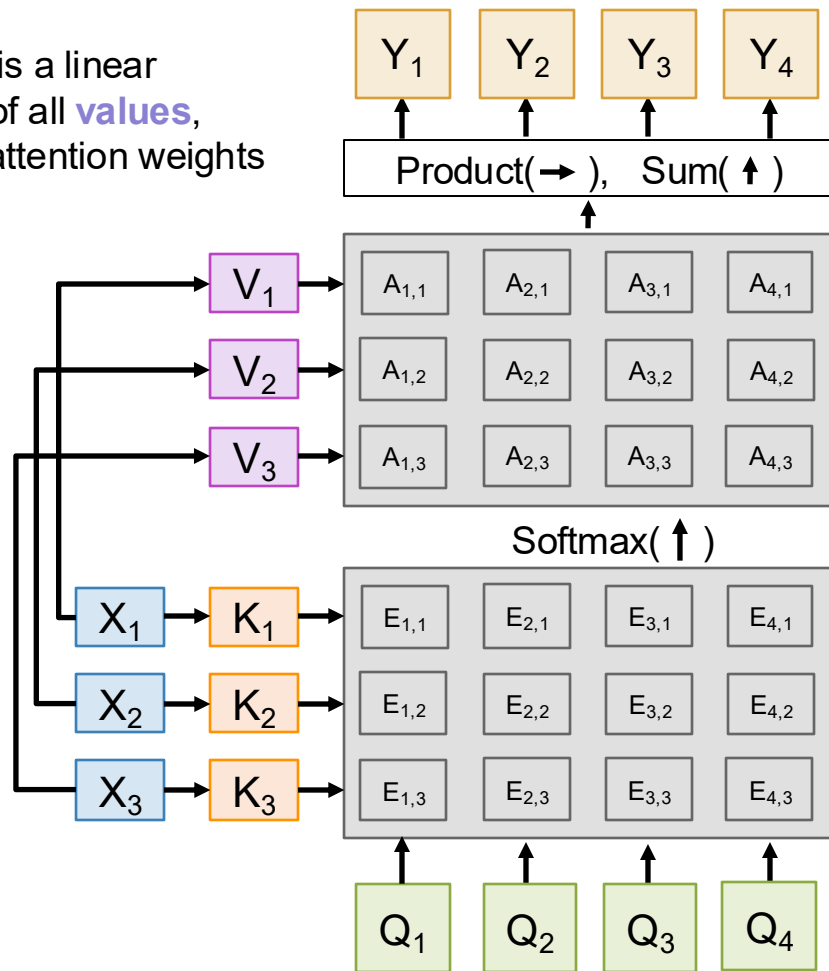
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$

Each **output** is a linear combination of all **values**, weighted by attention weights



Cross-Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Each **query** produces one **output**, which is a mix of information in the **data** vectors

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

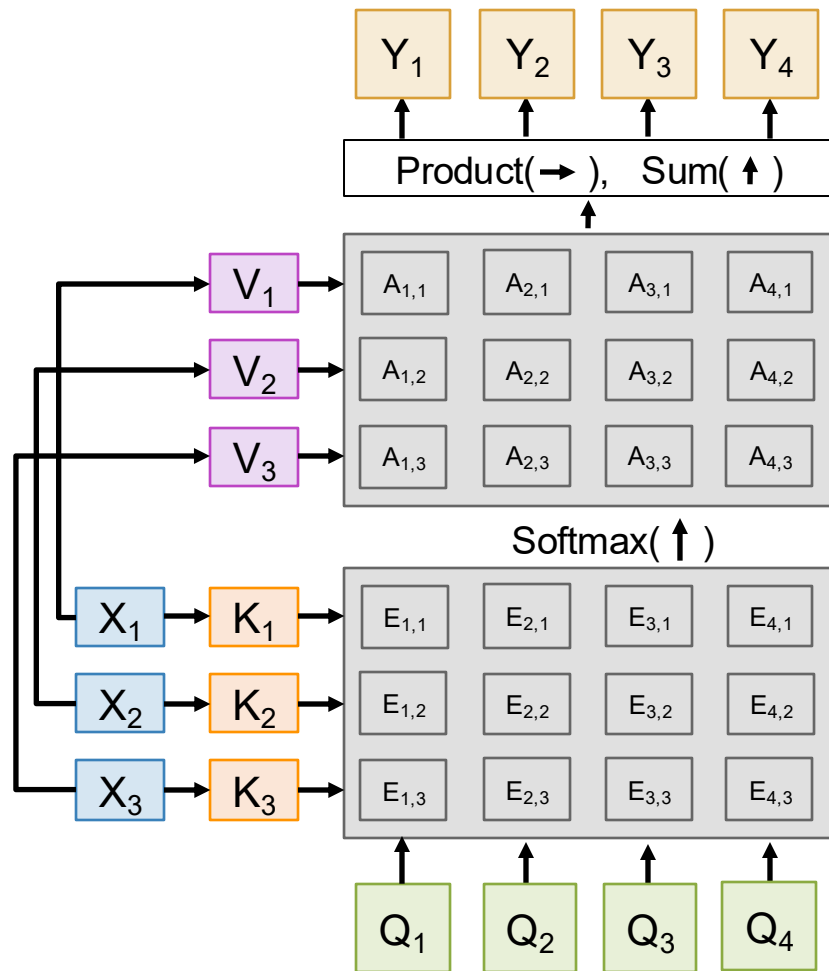
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

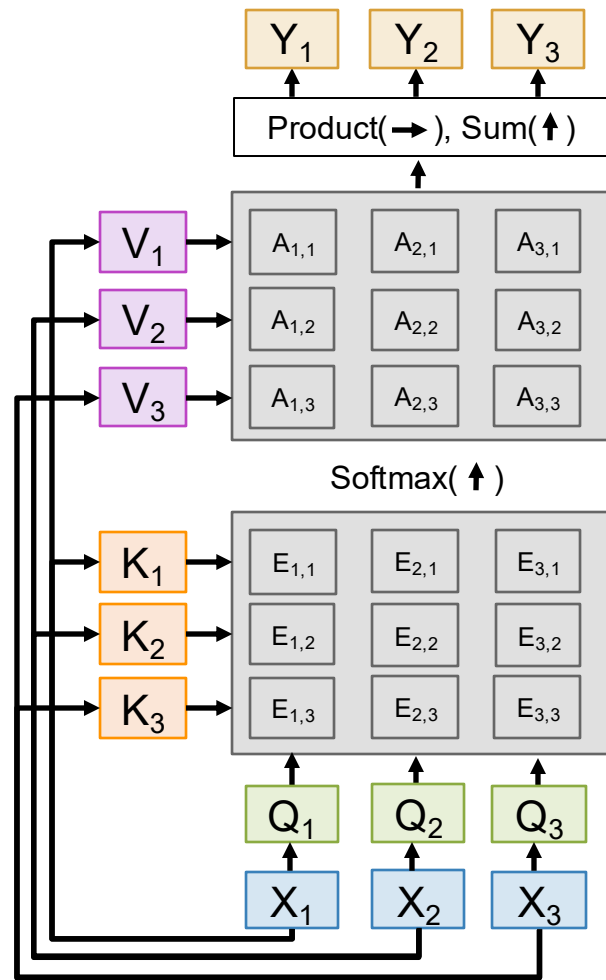
Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Each **input** produces one **output**, which is a mix of information from all **inputs**

Shapes get a little simpler:

- N input vectors, each D_{in}
- Almost always $D_Q = D_V = D_{out}$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

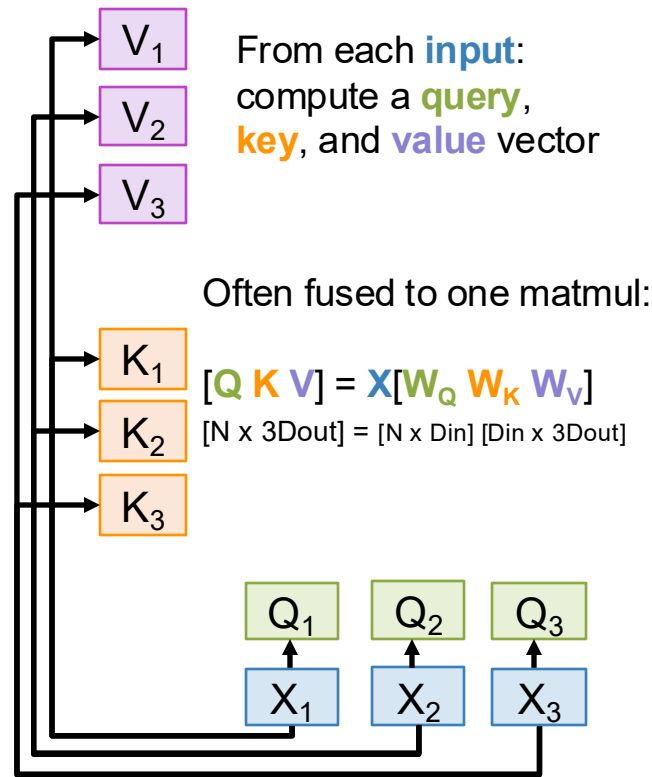
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

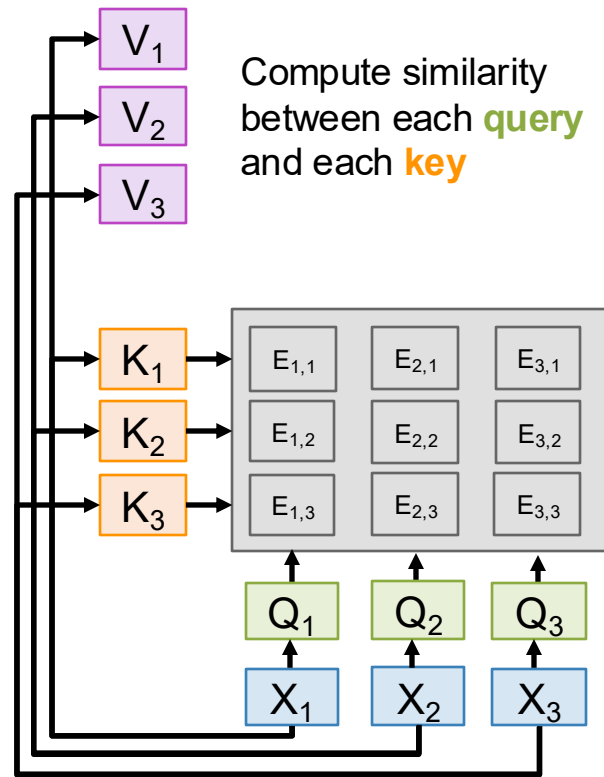
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

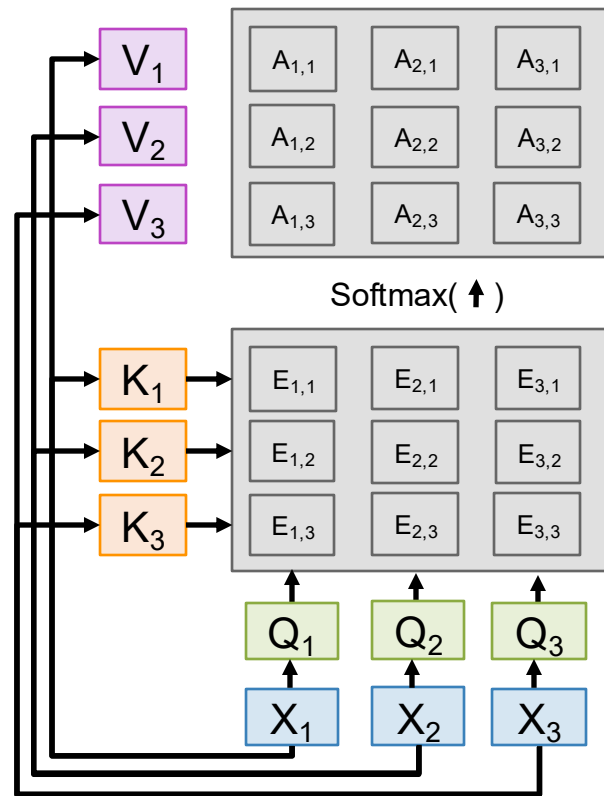
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Normalize over each column:
each **query** computes a distribution over **keys**



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

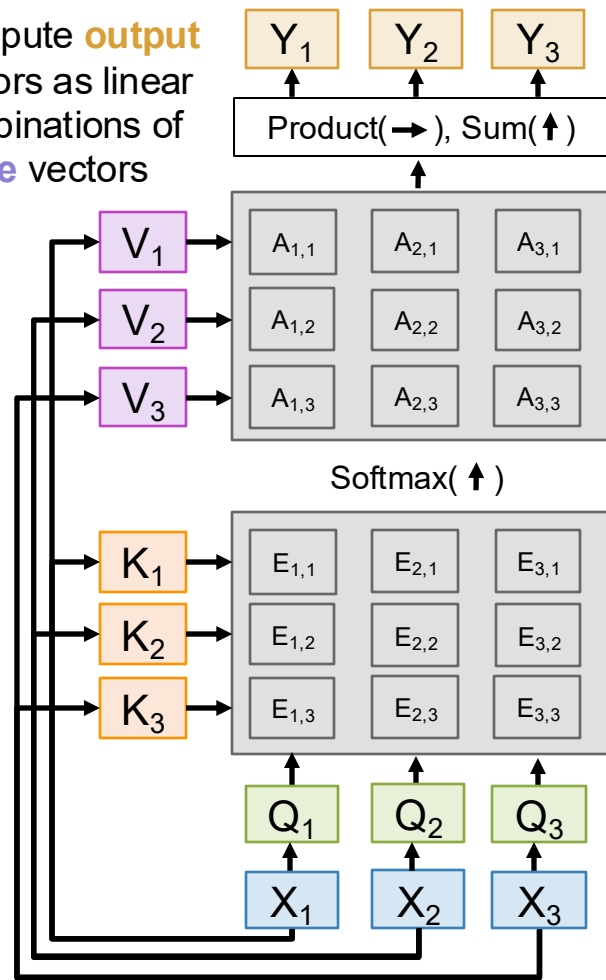
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Compute **output** vectors as linear combinations of **value** vectors



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

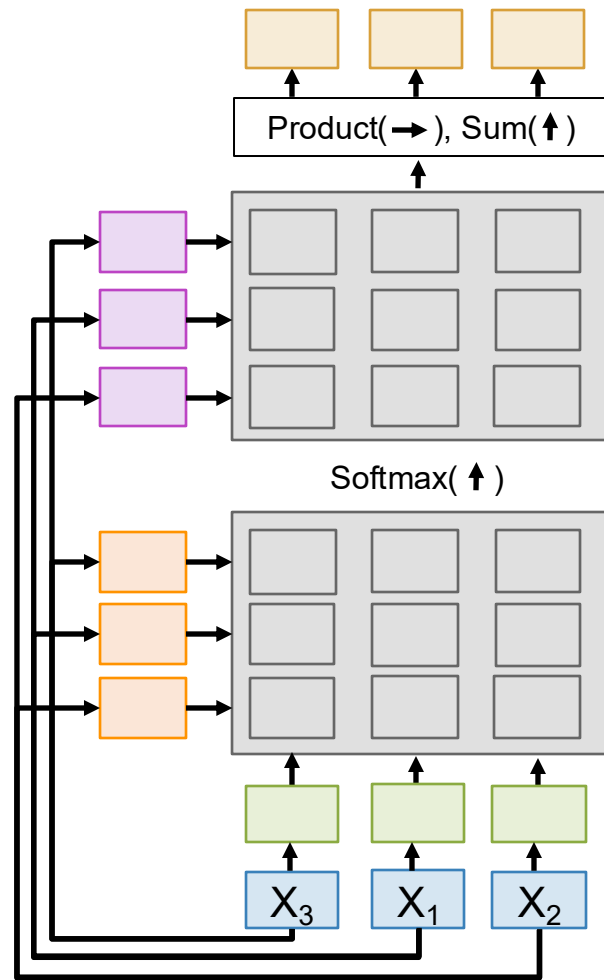
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Consider permuting **inputs**:



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

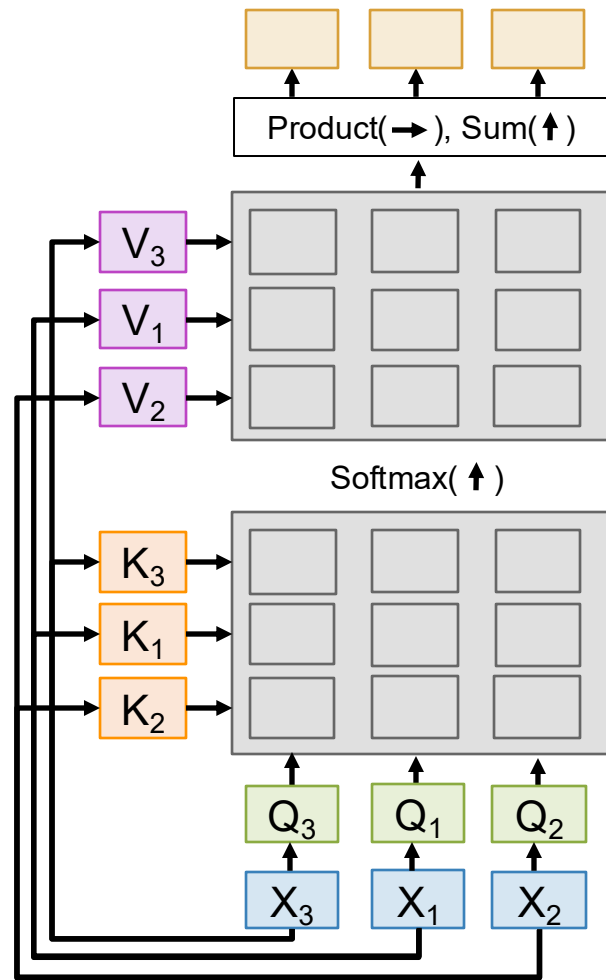
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Consider permuting **inputs**:

Queries, **keys**, and **values**
will be the same but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

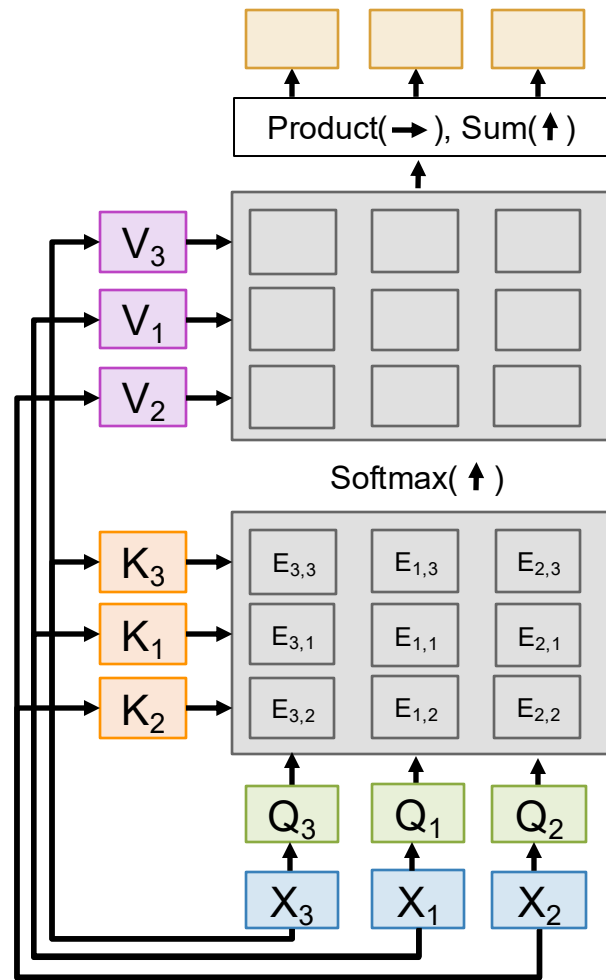
Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Consider permuting **inputs**:

Queries, **keys**, and **values**
will be the same but permuted

Similarities are the same but
permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

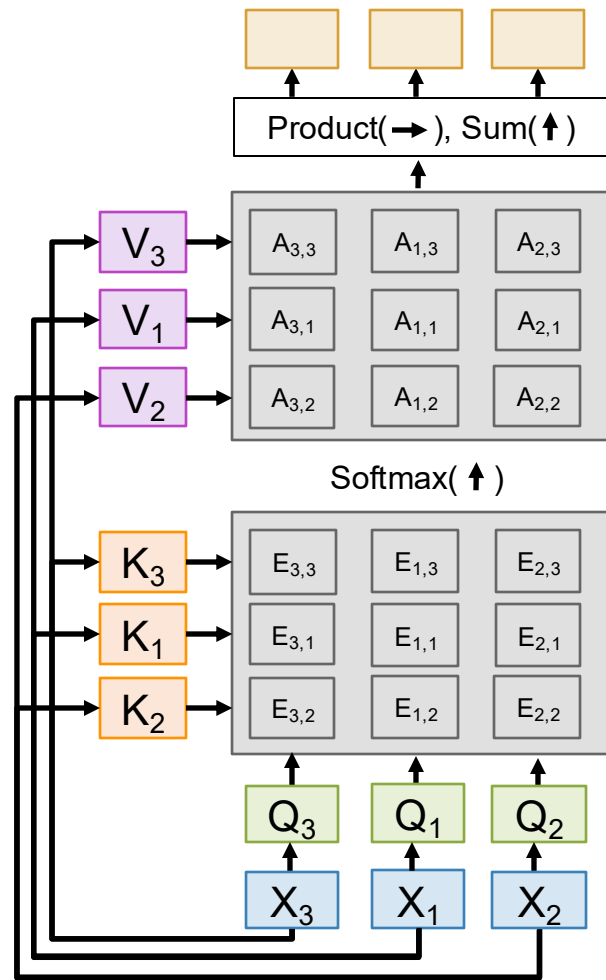
$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted

Attention weights are the same but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

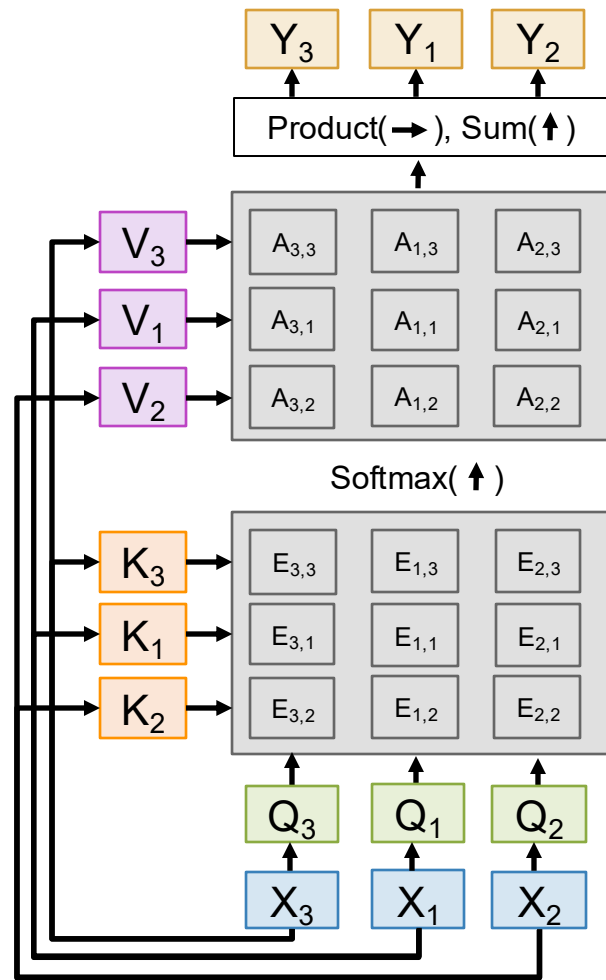
Consider permuting **inputs**:

Queries, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted

Attention weights are the same but permuted

Outputs are the same but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

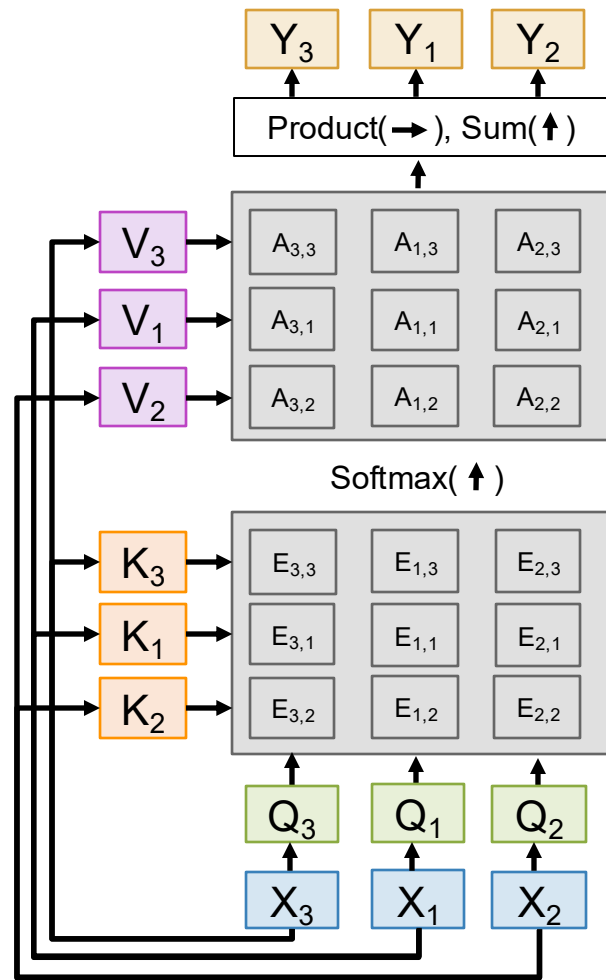
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Self-Attention is
permutation equivariant:
 $F(\sigma(X)) = \sigma(F(X))$

This means that Self-Attention
works on **sets of vectors**



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Problem: Self-Attention does not know the order of the sequence

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

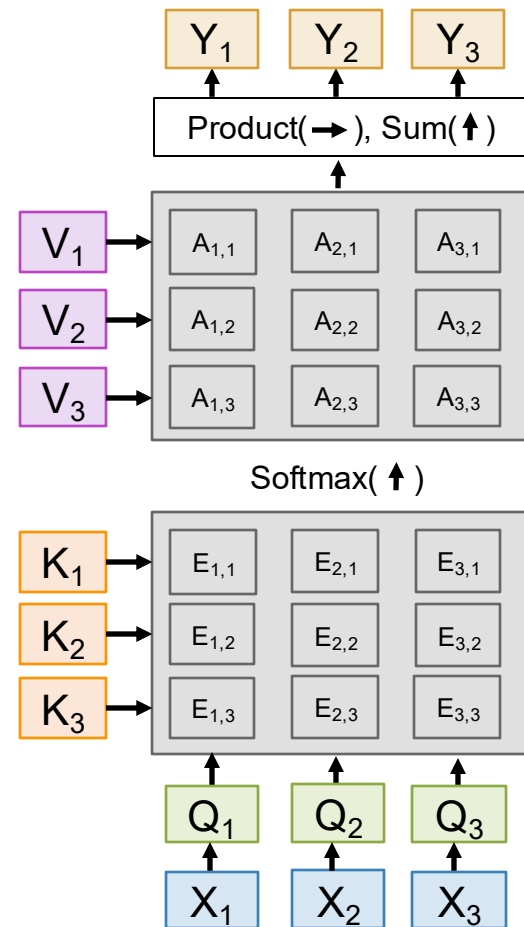
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

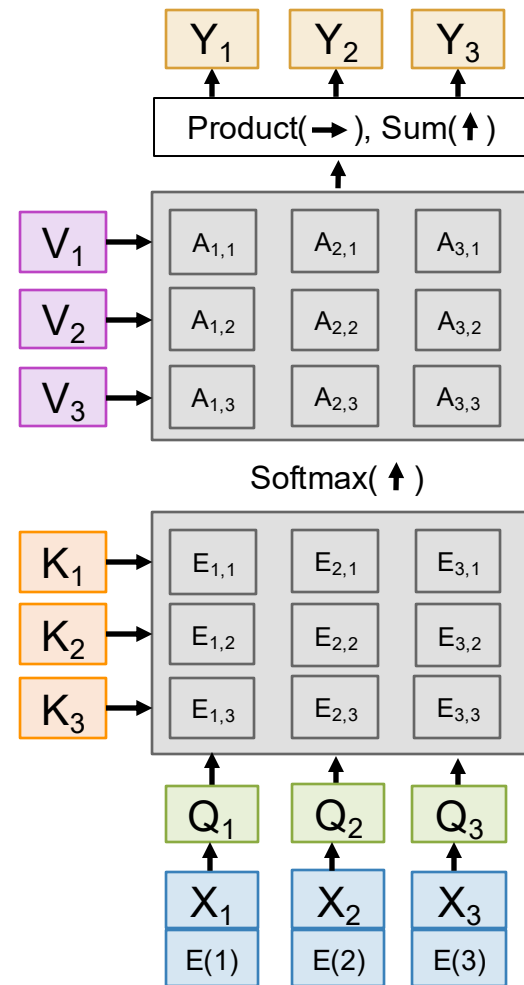
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Problem: Self-Attention does not know the order of the sequence

Solution: Add positional encoding to each input; this is a vector that is a fixed function of the index



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D_{in}]$

Key matrix: $\mathbf{W}_K$ $[D_{in} \times D_{out}]$

Value matrix: $\mathbf{W}_V$ $[D_{in} \times D_{out}]$

Query matrix: $\mathbf{W}_Q$ $[D_{in} \times D_{out}]$

Override similarities with $-\infty$;
this controls which inputs each
vector is allowed to look at.

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[N \times D_{out}]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[N \times D_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[N \times D_{out}]$

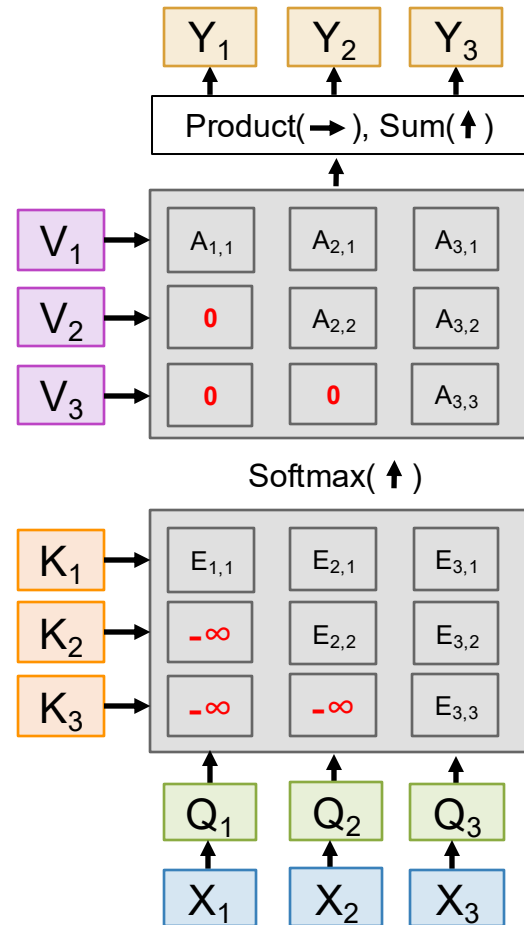
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[N \times N]$

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ $[N \times N]$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times D_{out}]$

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: X $[N \times D_{in}]$

Key matrix: W_K $[D_{in} \times D_{out}]$

Value matrix: W_V $[D_{in} \times D_{out}]$

Query matrix: W_Q $[D_{in} \times D_{out}]$

Computation:

Queries: $Q = XW_Q$ $[N \times D_{out}]$

Keys: $K = XW_K$ $[N \times D_{out}]$

Values: $V = XW_V$ $[N \times D_{out}]$

Similarities: $E = QK^T / \sqrt{D_Q}$ $[N \times N]$

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

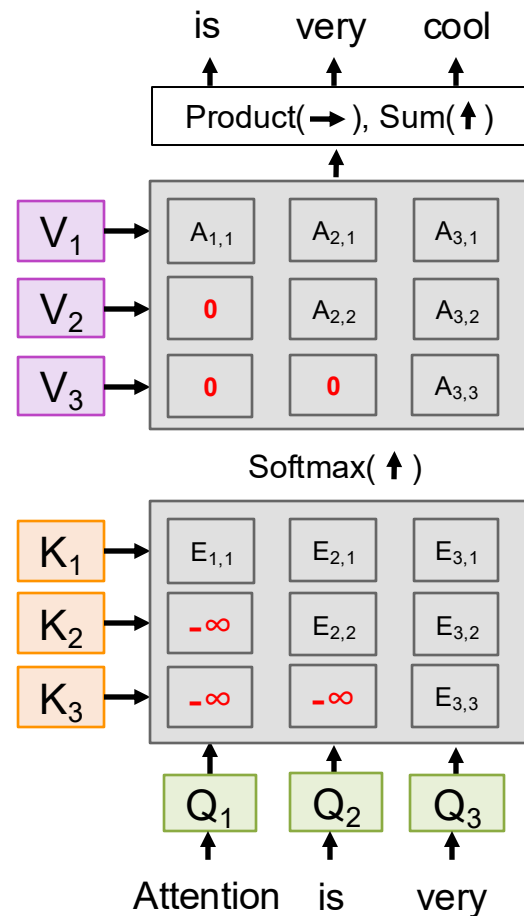
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ $[N \times N]$

Output vector: $Y = AV$ $[N \times D_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

Override similarities with $-\infty$; this controls which inputs each vector is allowed to look at.

Used for language modeling where you want to predict the next word



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D_{in}]$

Key matrix: $\mathbf{W}_K$ $[D_{in} \times D_{out}]$

Value matrix: $\mathbf{W}_V$ $[D_{in} \times D_{out}]$

Query matrix: $\mathbf{W}_Q$ $[D_{in} \times D_{out}]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[N \times D_{out}]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[N \times D_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[N \times D_{out}]$

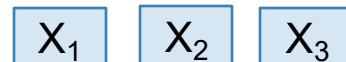
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[N \times N]$

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ $[N \times N]$

Output vector: $\mathbf{Y} = \mathbf{AX}$ $[N \times D_{out}]$

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

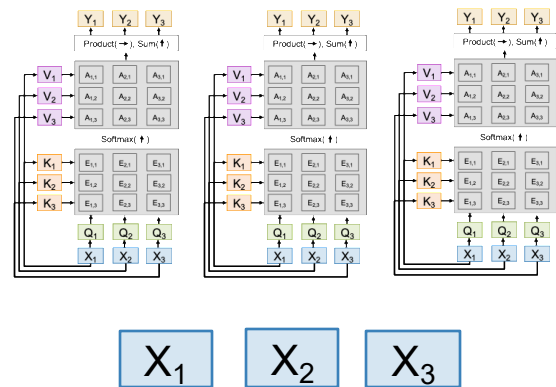
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AX}$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

H = 3 independent
self-attention layers
(called heads), each
with their own weights



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_K [$D_{in} \times D_{out}$]

Value matrix: W_V [$D_{in} \times D_{out}$]

Query matrix: W_Q [$D_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_K$ [$N \times D_{out}$]

Values: $V = XW_V$ [$N \times D_{out}$]

Similarities: $E = QK^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

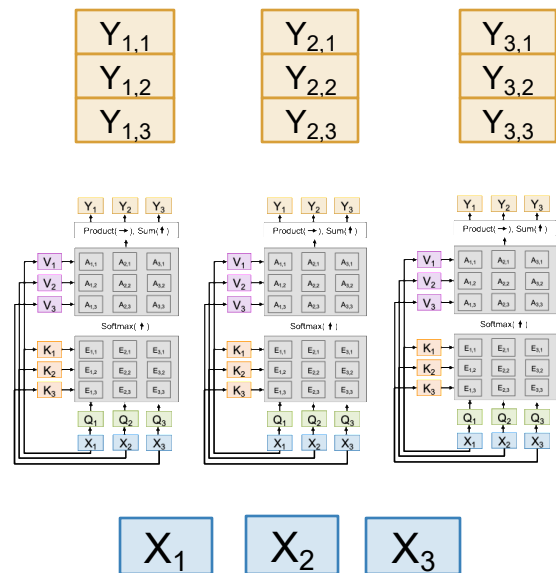
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ [$N \times N$]

Output vector: $Y = AX$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Stack up the H independent outputs for each input X

H = 3 independent self-attention layers (called heads), each with their own weights



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: X [$N \times D_{in}$]

Key matrix: W_K [$D_{in} \times D_{out}$]

Value matrix: W_V [$D_{in} \times D_{out}$]

Query matrix: W_Q [$D_{in} \times D_{out}$]

Computation:

Queries: $Q = XW_Q$ [$N \times D_{out}$]

Keys: $K = XW_K$ [$N \times D_{out}$]

Values: $V = XW_V$ [$N \times D_{out}$]

Similarities: $E = QK^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ [$N \times N$]

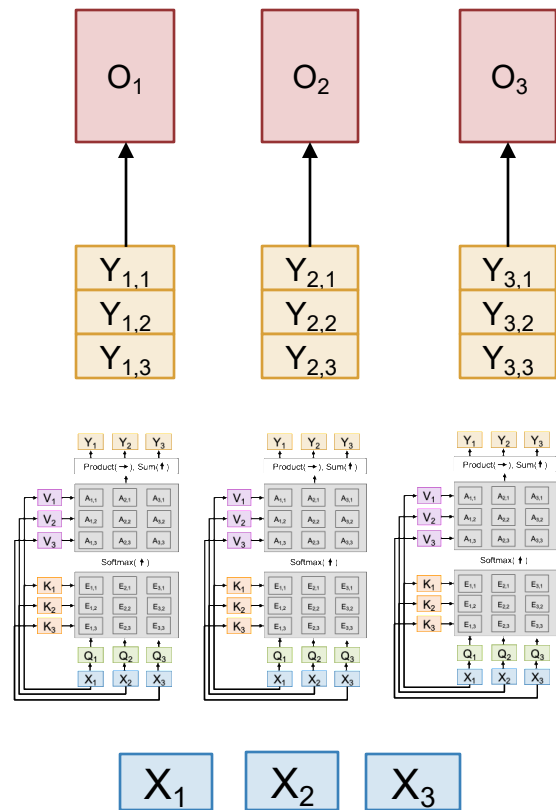
Output vector: $Y = AX$ [$N \times D_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$

Output projection fuses data from each head

Stack up the H independent outputs for each input X

H = 3 independent self-attention layers (called heads), each with their own weights



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $\mathbf{X}$ [N x D]

Key matrix: $\mathbf{W}_K$ [D x HD_H]

Value matrix: $\mathbf{W}_V$ [D x HD_H]

Query matrix: $\mathbf{W}_Q$ [D x HD_H]

Output matrix: $\mathbf{W}_O$ [HD_H x D]

Each of the H parallel layers use a qkv dim of D_H = “head dim”

Usually $D_H = D / H$, so inputs and outputs have the same dimension

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$H \times N \times D_H$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$H \times N \times D_H$]

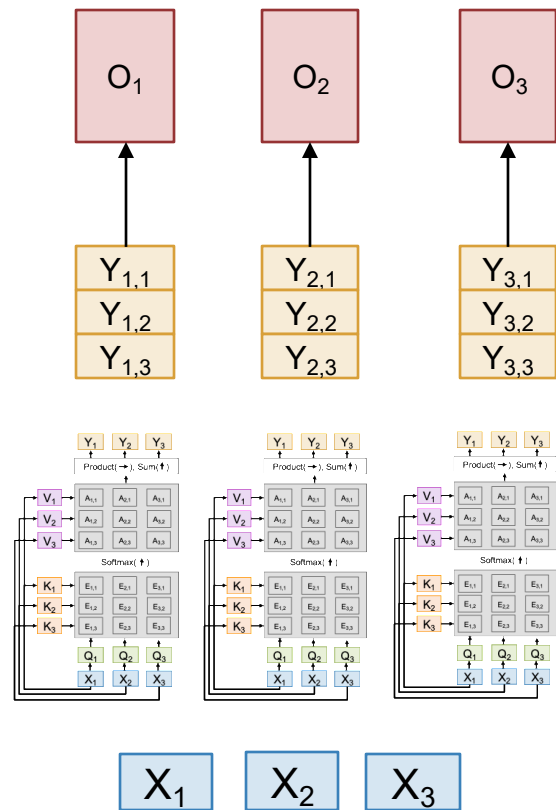
Values: $\mathbf{V} = \mathbf{XW}_V$ [$H \times N \times D_H$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$H \times N \times N$]

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ [$H \times N \times N$]

Head outputs: $\mathbf{Y} = \mathbf{AV}$ [$H \times N \times D_H$] $\Rightarrow$ [$N \times HD_H$]

Outputs: $\mathbf{O} = \mathbf{YW}_O$ [$N \times D$]



Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D$]

Key matrix: $\mathbf{W}_K$ [$D \times HD_H$]

Value matrix: $\mathbf{W}_V$ [$D \times HD_H$]

Query matrix: $\mathbf{W}_Q$ [$D \times HD_H$]

Output matrix: $\mathbf{W}_O$ [$HD_H \times D$]

In practice, compute all H heads in parallel using batched matrix multiply operations.

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$H \times N \times D_H$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$H \times N \times D_H$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$H \times N \times D_H$]

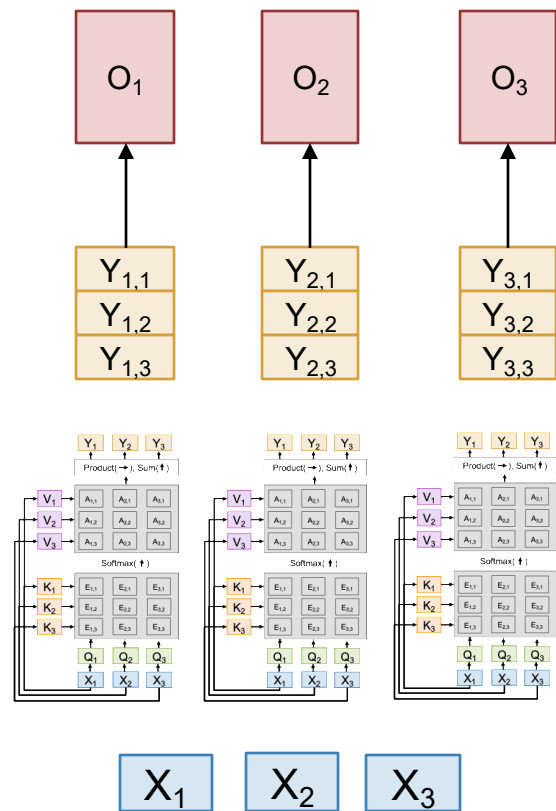
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$H \times N \times N$]

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ [$H \times N \times N$]

Head outputs: $\mathbf{Y} = \mathbf{AV}$ [$H \times N \times D_H$] $\Rightarrow$ [$N \times HD_H$]

Outputs: $\mathbf{O} = \mathbf{YW}_O$ [$N \times D$]

Used everywhere in practice.



Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Q: How much compute does this take as the number of vectors N increases?

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Q: How much compute does this take as the number of vectors N increases?

A: $O(N^2)$

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Q: How much memory does this take as the number of vectors N increases?

Self-Attention is Four Matrix Multiplies!

Inputs:

Input vectors: $\mathbf{X}$ $[N \times D]$

Key matrix: $\mathbf{W}_K$ $[D \times HD_H]$

Value matrix: $\mathbf{W}_V$ $[D \times HD_H]$

Query matrix: $\mathbf{W}_Q$ $[D \times HD_H]$

Output matrix: $\mathbf{W}_O$ $[HD_H \times D]$

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[H \times N \times D_H]$

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[H \times N \times D_H]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[H \times N \times D_H]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ $[H \times N \times N]$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ $[H \times N \times N]$

Head outputs: $\mathbf{Y} = \mathbf{AV}$ $[H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $\mathbf{O} = \mathbf{YW}_O$ $[N \times D]$

1. QKV Projection

$[N \times D]$ $[D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H]$ $[H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N]$ $[H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ $[HD_H \times D] \Rightarrow [N \times D]$

Q: How much memory does this take as the number of vectors N increases?

A: $O(N^2)$

Self-Attention is Four Matrix Multiplies!

If $N=100K$, $H=64$ then
 $H \times N \times N$ attention weights
take 1.192 TB! GPUs don't
have that much memory...

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D$]

Key matrix: $\mathbf{W}_K$ [$D \times HD_H$]

Value matrix: $\mathbf{W}_V$ [$D \times HD_H$]

Query matrix: $\mathbf{W}_Q$ [$D \times HD_H$]

Output matrix: $\mathbf{W}_O$ [$HD_H \times D$]

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$H \times N \times D_H$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$H \times N \times D_H$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$H \times N \times D_H$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$H \times N \times N$]

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ [$H \times N \times N$]

Head outputs: $\mathbf{Y} = \mathbf{AV}$ [$H \times N \times D_H$] $\Rightarrow$ [$N \times HD_H$]

Outputs: $\mathbf{O} = \mathbf{YW}_O$ [$N \times D$]

1. QKV Projection

$[N \times D]$ [$D \times 3HD_H$] $\Rightarrow$ [$N \times 3HD_H$]

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of
shape [$H \times N \times D_H$]

2. QK Similarity

$[H \times N \times D_H]$ [$H \times D_H \times N$] $\Rightarrow$ [$H \times N \times N$]

3. V-Weighting

$[H \times N \times N]$ [$H \times N \times D_H$] $\Rightarrow$ [$H \times N \times D_H$]

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ [$HD_H \times D$] $\Rightarrow$ [$N \times D$]

Q: How much memory does this take
as the number of vectors N increases?

A: $O(N^2)$

Self-Attention is Four Matrix Multiplies!

If $N=100K$, $H=64$ then
 $H \times N \times N$ attention weights
take 1.192 TB! GPUs don't
have that much memory...

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D$]

Key matrix: $\mathbf{W}_K$ [$D \times HD_H$]

Value matrix: $\mathbf{W}_V$ [$D \times HD_H$]

Query matrix: $\mathbf{W}_Q$ [$D \times HD_H$] Makes large N

Output matrix: $\mathbf{W}_O$ [$HD_H \times D$] possible

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$H \times N \times D_H$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$H \times N \times D_H$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$H \times N \times D_H$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$H \times N \times N$]

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=2)$ [$H \times N \times N$]

Head outputs: $\mathbf{Y} = \mathbf{AV}$ [$H \times N \times D_H$] $\Rightarrow$ [$N \times HD_H$]

Outputs: $\mathbf{O} = \mathbf{YW}_O$ [$N \times D$]

Flash Attention
algorithm computes
2+3 at the same time
without storing the
full attention matrix!

1. QKV Projection

$[N \times D]$ [$D \times 3HD_H$] $\Rightarrow$ [$N \times 3HD_H$]

Split and reshape to get $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ each of
shape [$H \times N \times D_H$]

2. QK Similarity

$[H \times N \times D_H]$ [$H \times D_H \times N$] $\Rightarrow$ [$H \times N \times N$]

3. V-Weighting

$[H \times N \times N]$ [$H \times N \times D_H$] $\Rightarrow$ [$H \times N \times D_H$]

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H]$ [$HD_H \times D$] $\Rightarrow$ [$N \times D$]

Q: How much memory does this take
as the number of vectors N increases?

A: $O(N)$ with Flash Attention

Dao et al, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", 2022